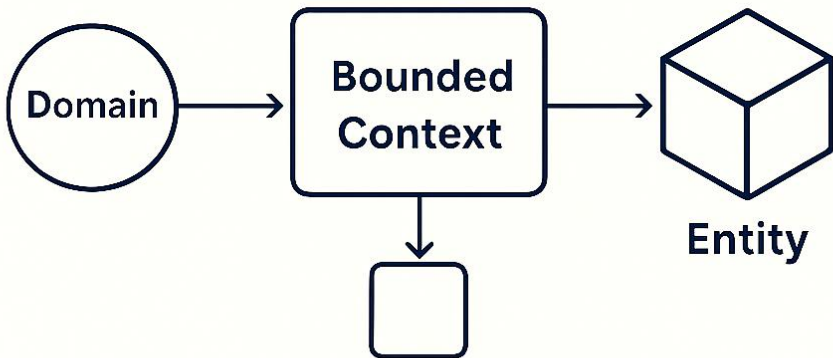


Answer 11-15

Top 100

DDD

Questions



فهرست

1.DDD چیست؟ 1.
2. چرا به جای معماری لایه ای سنتی از DDD استفاده کنیم؟ 2.
3. چه پروژه هایی می توانند از DDD استفاده کنند؟ 3.
4. عناصر اصلی تشکیل دهنده DDD چیست؟ 4.
5. Entity چیست؟ 4.
6. Value Object چیست؟ 5.
7. Aggregate چیست و چرا به آن نیاز داریم؟ 6.
8. Aggregate Root چیست؟ 7.
9. Repository چیست؟ 8.
10. Factory چیست؟ 9.
11. Domain Service چیست؟ 10.
12. Bounded Context چیست؟ 12.
13. Ubiquitous Language چیست؟ 14.
14. تفاوت بین Domain Model و Data Model چیست؟ 15.
15. تفاوت Domain Logic و Application Logic چیست؟ 16.

1. DDD چیست؟

DDD مخفف عبارت Domain Driven Design به معنی طراحی دامنه محور است. DDD معماری نیست بلکه یک روش فکری است که برای کمک به طراحی و پیاده سازی پروژه های نرم افزاری با دامنه پیچیده ساخته شده است. تمام تمرکز این روش فکری بر روی مدل سازی کسب و کار و حل پیچیدگی های قوانین کسب و کار در پروژه نرم افزاری است. استفاده از یک زبان مشترک بین تیم فنی و تیم تحلیل، ارتباط مداوم تیم فنی و تیم تحلیل و اعمال بیشترین تلاش برای درک عمیق تر دامنه کسب و کار، از توصیه های مهم DDD است.

2. چرا به جای معماری لایه ای سنتی از DDD استفاده

کنیم؟

در معماری لایه ای قدیم معمولاً پروژه بر اساس **وظایف** به چند لایه دیگر تقسیم می‌شد. یک لایه رابط کاربری نرم افزار با دنیای بیرون، یک لایه مخصوص پیاده سازی قوانین کسب و کار و یک لایه هم مخصوص ارتباط با دیتابیس بود. در این نوع طراحی، **مدل کسب و کار** به **شکل دیتا** بود، یعنی مدل نه رفتار داشت و نه کامل بود بلکه فقط به صورت یک سری دیتا بود که در دیتابیس زندگی می‌کرد و در مواقع نیاز بین لایه های مختلف نرم افزار جا به جا می‌شد. اما در DDD تمام تمرکز بر روی مدلی از کسب و کار است که تمام **قوانین کسب و کار** را با خود به همراه دارد و علاوه بر آن تمام **رفتارهایی** که در دنیای واقعی می‌تواند داشته باشد را درون خود **پیاده سازی** کرده است. حال ممکن است ساختار این مدل با مدل دیتابیس نیز متفاوت باشد چون به مدل به چشم یک **موجودیت** نگاه می‌کنیم نه دیتا.

در نتیجه می‌توان گفت در معماری سنتی لایه ای تمرکز بر روی **وظیفه** هر لایه و **تکنولوژی** مورد استفاده در پروژه بود و از مدل فقط به عنوان ابزاری برای رد و بدل کردن دیتا استفاده می‌شد. اما در DDD سعی می‌کنیم تمام **موجودیت** های کسب و کار را **شناسایی** کنیم و با پیاده سازی **قوانین کسب و کار** و **رفتارهای** منحصر به فرد آنها درون آنها **پیچیدگی** های کسب و کار را از بین ببریم.

3. چه پروژه هایی می‌توانند از DDD استفاده کنند؟

اگر در پروژه نرم افزاری با کسب و کاری روبرو هستید که:

- قوانین پیچیده دارد و دانش تیم درباره این قوانین مدام عمیق تر می‌شود
- نیازهای کسب و کار ممکن است در آینده تغییرات زیاد داشته باشد
- شرایط بازار به سرعت تغییر می‌کند و روی کسب و کار تاثیر می‌گذارد
- بین موجودیت های کسب و کار روابط پیچیده وجود دارد
- نیاز به یک پروژه نرم افزاری دارید که در درازمدت قابل نگهداری و توسعه باشد

بهتر است از DDD بهره ببرید، چرا که تمرکز آن بر روی قلب کسب و کار است و تمام اجزای پروژه را طوری طراحی می‌کند که همه در خدمت قلب کسب و کار که همان مدل دامنه است، باشند.

اما اگر با کسب و کاری روبرو هستید که:

- قوانین خیلی ساده ای دارد
- شرایط آن دچار تغییرات زیاد نمی‌شود
- تمامی فرآیند های آن شامل یک سری مراحل ساده و پشت سر هم هستند
- به صورت مقطعی و کوتاه مدت نیاز به نرم افزار دارد
- نیاز به یک نرم افزار قدرتمند ندارد و فقط نیاز به یک نمونه اولیه دارد

بهتر است از همان مدل سنتی معماری لایه ای استفاده کنید.

4. عناصر اصلی تشکیل دهنده DDD چیست؟

در طراحی استراتژیک عناصر اصلی شامل Subdomain, Bounded Context, Ubiquitous Language, Context Map, Bounded Context Relationship هستند.

در طراحی فنی عناصر اصلی شامل: لایه بندی پروژه، Entity, Aggregate, Value Object, Service, Repository, Factory, Specification Pattern, Design Patterns, Module هستند.

5. Entity چیست؟

در هر کسب و کاری یک سری موجودیت ها وجود دارند. برخی از آنها برای خدمت رسانی کسب و کار به مشتریان الزامی هستند. تمام این موجودیت های ضروری برای حیات کسب و کار باید مدل سازی شوند و در پروژه نرم افزاری پیاده سازی شوند. برخی از این موجودیت های ضروری برای حیات کسب و کار به واسطه یک شناسه منحصر به فرد شناخته می شوند و از دیگر موجودیت ها از همان جنس متمایز می شوند. هر Entity یک سری مشخصات دارد که همیشه این مشخصات را با خود به همراه دارد. تمام مشخصات هر Entity بارها می تواند در طول عمر پروژه تغییر کنند اما شناسه آنها همیشه ثابت است و هیچ گاه تغییر نمی کند.

برای مثال در یک فروشگاه اینترنتی، موجودیت مشتری یک Entity حساب می شود زیرا هر مشتری با مشتری دیگر متمایز است یعنی حتی اگر نام آنها یکسان باشند باز هم دو مشتری متفاوت حساب می شوند.

6. Value Object چیست؟

یک سری موجودیت ها در هر کسب و کاری هستند که هویت ندارند، یعنی شما نمی‌توانید شناسه منحصر به فردی برای آنها در نظر بگیرید، چرا که در مورد آنها تنها چیزی که اهمیت دارد مقدار آنها است. به فرض مثال در فروشگاه های اینترنتی موجودیتی به نام آدرس مشتری وجود دارد که مشخص می‌کند مشتری در کدام استان و کدام شهر زندگی می‌کند. برای شما اهمیتی ندارد که این موجودیت دارای شناسه باشد، تنها چیزی که درباره آن اهمیت دارد نام استان و نام شهر است. به هر موجودیتی که فقط مقدار آن برای ما اهمیت داشته باشد Value Object گفته می‌شود.

Value Object ها با مقدارشان با یکدیگر مقایسه می‌شوند. پس هر دو Value Object ایی که مقدار یکسان داشته باشند با یکدیگر برابر هستند. از آنجایی که در این نوع موجودیت ها به واسطه مقدارشان از یکدیگر متمایز می‌شوند، بنابراین در طول عمر خود هرگز نباید تغییر کنند تا یکپارچگی مدل و صحت داده ها در سیستم حفظ شود. اگر در قسمتی از یک فرآیند مقدار یک Value Object نیاز به تغییر داشته باشد، مقدار Value Object را تغییر نمی‌دهید بلکه یک شی جدید از آن Value Object با مقدار جدید می‌سازید و فرآیند در ادامه مسیر خود با Value Object جدید کار خواهد کرد.

7. Aggregate چیست و چرا به آن نیاز داریم؟

در هر کسب و کاری تعداد زیاد موجودیت از نوع Entity و Value Object وجود دارد. بسیاری از اوقات برخی از این موجودیت ها به تنهایی کامل نیستند و نیاز به حضور دیگر موجودیت ها دارند. مثلا در یک فروشگاه اینترنتی موجودیت Order به تنهایی کامل نیست و برای پردازش یک سری فرآیندها نیاز به حضور موجودیت OrderItem دارد. هر بار که یک OrderItem جدید برای یک Order ساخته شود، موجودیت Order باید خود را بروزرسانی کند. هر بار که یک OrderItem ویرایش شود، موجودیت Order یک بار باید تمام قوانین را مجدد چک کند تا در صورت اطمینان از رعایت شدن تمام قوانین اجازه ثبت تغییرات را بدهد. پس دو موجودیت Order و OrderItem همیشه با یکدیگر زندگی می‌کنند پس این دو موجودیت را داخل یک مرز فرضی قرار می‌دهیم.

به مرز فرضی که موجودیت هایی وابسته به یکدیگر را داخل خود نگهداری می‌کند Aggregate گفته می‌شود. هر تغییری داخل این مرز باید داخل یک تراکنش انجام شود تا در صورت نیاز به تغییر چندین موجودیت داخل Aggregate یا همه با هم تغییر کنند یا هیچ کدام تغییر نکنند و همچنین قبل از تایید تراکنش یک بار تمام قوانین داخل مرز یک بار چک شود و در صورت رعایت شدن تمامی آنها تغییرات اعمال شوند.

8. Aggregate Root چیست؟

داخل هر Aggregate یک Entity به عنوان Aggregate Root شناخته می‌شود. از آنجایی که فضای داخل Aggregate همیشه باید یکپارچه باشد و هیچ گاه نباید قوانین داخل آن نقض شوند بنابراین هیچ فرآیندی خارج از Aggregate نمی‌تواند به موجودیت های داخل آن دسترسی داشته باشد، بلکه فقط می‌تواند به Aggregate Root دسترسی داشته باشد. Aggregate Root پل ارتباطی بین دنیای بیرون Aggregate با دنیای درون آن است و هر فرآیند و یا هر شی ای که با داخل Aggregate کار داشته باشد باید با Aggregate Root تعامل کند.

اجرای تراکنش های داخل Aggregate و تایید یا رد آن به عهده Aggregate Root است. چک کردن قوانین داخل Aggregate و جلوگیری از نقض آنها به عهده Aggregate Root است.

9. Repository چیست؟

هر نرم افزاری برای نگهداری دیتا از یک یا چند دیتابیس استفاده می‌کند. برای آنکه در هر قسمت از نرم افزار که نیاز به عملیات خواندن و نوشتن دیتا داشتیم مستقیم به دیتابیس وصل نشویم و لایه ها را وابسته به تکنولوژی دیتابیس نکنیم، DDD الگوی Repository را برای تعامل به دیتابیس معرفی کرده است. Repository یک الگوی طراحی برای تعامل با دیتابیس است که هر آن چیزی که برای تعامل با دیتابیس نیاز داشته باشید داخل آن پیاده سازی می‌کنید و لایه های Domain و Application برای فراخوانی سرویس های Repository از Interface آن استفاده می‌کنند تا به تکنولوژی دیتابیس وابسته نباشند.

تمام Entity ها نمی‌توانند Repository داشته باشند بلکه فقط Aggregate ها اجازه دارند Repository داشته باشند و در صورتی که نیاز به خواندن یا نوشتن دیتای Entity های داخل Aggregate داشته باشید باید از طریق Repository همان Aggregate استفاده کنید. اینگونه هر بار برای خواندن و نوشتن دیتا داخل دیتابیس Aggregate یک بار می‌تواند تمام قوانین را صحت سنجی کند و سپس در صورت رعایت قوانین اجازه بارگزاری دیتا یا نوشتن دیتا را بدهد.

10. Factory چیست؟

Factory در معنا یعنی کارخانه. جایی که مواد اولیه را در ورودی دریافت کرده و محصول نهایی از آن خارج می‌شود. گاهی لازم است برای ساخت یک Entity جدید یک سری پردازش‌های جزئی انجام شود یا برای ساخت یک Aggregate جدید نیاز است چندین Entity جدید ساخته شود یا تمام قوانین مربوطه چک شوند. همه این موارد باعث می‌شود عملیات ساخت شی جدید پیچیده شود. پس برای کپسوله سازی این پیچیدگی از الگوی Factory استفاده می‌کنیم، الگویی که در ظاهر چند ورودی ساده می‌گیرد و یک شی نهایی را تحویل می‌دهد اما در باطن برای ساخت آن شی نهایی، داخل خود یک سری عملیات پیچیده را انجام می‌دهد و Client هیچ گاه از جزئیات آن باخبر نمی‌شود.

11. Domain Service چیست؟

در هر کسب و کاری یک سری **موجودیت** وجود دارد که می‌توانند یکی از سه نوع Entity, Value Object یا Aggregate باشند. تمام رفتارهای کسب و کار توسط این **موجودیت** ها **پیاده سازی** می‌شوند.

مثال یک) اگر در پروژه یک Value Object برای آدرس مشتری در نظر گرفته اید، هر بار که نیاز به تغییر آدرس مشتری داشته باشید به Value Object مربوط به آدرس درخواست می‌دهید یک نمونه جدید با مقدار جدید از خودش بسازد و به خروجی برگرداند. در واقع رفتار ساخت یک آدرس جدید متعلق به Value Object آدرس است و خودش این کار را انجام می‌دهد.

مثال دو) اگر در پروژه یک Entity برای مشتری در نظر گرفته اید و مشتری نیاز به تایید شماره همراه خود داشته باشد این رفتار را در خود Entity پیاده سازی می‌کنید.

پس تمام **فرآیندها** و **رفتارهایی** که در کسب و کار وجود دارد و **متعلق** به یک **موجودیت** مشخص است باید **داخل** همان **موجودیت** پیاده سازی شود. اما گاهی اوقات فرآیندهایی وجود دارند که متعلق به هیچ موجودیت خاصی نیست و معمولاً برای انجام شدن نیاز به چند Entity دارند، این رفتارها یا فرآیندها را به صورت **سرویس جداگانه** ای در پروژه پیاده سازی می‌کنیم.

مثال سه) فرض کنید به ازای هر کدام از پرداخت های مشتری باید مبلغی را تحت عنوان اعتبار به حساب مشتری اضافه کنیم. در اینجا دو موجودیت به نام های پرداخت و مشتری داریم ولی هیچ کدام مسئول این قانون کسب و کار

نیستند، پس یک سرویس پیاده سازی می‌کنیم که داخل آن هر دو موجودیت را به عنوان ورودی تحویل می‌دهیم و سپس اعتبار مشتری شارژ می‌شود.

به طور کلی هر فرآیند یا رفتاری که متعلق به یک موجودیت خاص نباشد کاندیدای مناسبی برای Domain Service بودن دارد.

یک تفاوت مهم دیگر Domain Service ها و Entity ها در نگهداری اطلاعات است. Entity ها از زمانی که در برنامه بارگزاری می‌شوند تا لحظه ای که از بین بروند دیتاهای مربوط به خود را نگهداری می‌کنند حتی اگر بار و بارها در فرآیند های مختلف مورد استفاده قرار بگیرند باز هم دیتای خود را به همراه دارند، اما، Domain Service ها هیچ دیتایی در خود نگهداری نمی‌کنند. آنها تمام اطلاعاتی که نیاز دارند را از ورودی دریافت کرده و پس از انجام پردازش ها و ارسال نتیجه به خروجی همه چیز را فراموش می‌کنند.

12. Bounded Context چیست؟

هر Bounded Context قسمت کوچکی از کسب و کار است. یک کسب و کار معمولاً قسمت‌های مختلفی دارد و هر قسمت مسئول یک سری از خدمات کسب و کار است.

هر کسب و کار خدمات متنوعی به مشتریان خود ارائه می‌دهد. تمام خدمات کسب و کار در کنار هم چرخ دنده‌های آن را تشکیل می‌دهند. برای آنکه کسب و کار بتواند نیاز مشتریان خود را برآورده کند و به درآمدزایی برسد باید تمام این چرخ دنده‌ها با هم حرکت کنند. هر چرخ دنده مسئول یک سری خدمات کسب و کار است. شکل هر چرخ دنده و ابعاد آن با بقیه متفاوت است ولی هیچ کدام به تنهایی نمی‌تواند حرکت کند بلکه تمام آنها باید به کمک یکدیگر به حرکت در بیایند.

هر چرخ دنده کسب و کار یک Bounded Context حساب می‌شود. Bounded Context‌ها در هر کسب و کار مرز فرضی هستند حول خدماتی که کسب و کار به مشتریان و کاربران خود ارائه می‌دهد. در واقع با این مرزبندی می‌توان خدماتی از کسب و کار که به یکدیگر مرتبط هستند را گروه بندی کرد.

برای مثال کسب و کار یک رستوران را در نظر بگیرید. یک قسمت از خدمات رستوران مربوط به ثبت سفارش مشتریان است و یک قسمت دیگر مربوط به مدیریت سفارشات توسط عوامل رستوران است. هر کدام را می‌توان به یک چرخ دنده تشبیه کرد و این یعنی هر کدام یک Bounded Context حساب می‌شوند و برای آنکه یک سفارش ثبت و به مرحله آماده سازی برسد باید این دو چرخ دنده با یکدیگر حرکت کنند.

هر Bounded Context مفاهیم و قوانین منحصر به فرد خدمات خود را دارد. برای هر Bounded Context یک پروژه جدا توسط تیم توسعه ایجاد می‌شود. پس بخشی از فضای Bounded Context ها مربوط به کسب و کار و قوانین آن است و بخشی دیگر مربوط به تیم توسعه و مسائل تکنیکال.

13. Ubiquitous Language چیست؟

Ubiquitous Language به معنی زبان مشترک بین افراد تیم تحلیل و افراد تیم توسعه است.

تیم تحلیل به زبان کسب و کار صحبت می‌کند. این افراد کسانی هستند که قوانین کسب و کار را به خوبی می‌شناسند، از طرفی تیم توسعه به زبان فنی صحبت می‌کند و تکنولوژی‌های مربوط به حوزه خود را می‌شناسد.

برای پیاده سازی صحیح پروژه نرم افزاری کسب و کار تیم تحلیل باید دانش مربوط به کسب و کار را با تیم توسعه به اشتراک بگذارد و تیم توسعه نیز باید ابزار محیط تکنیکال که به کسب و کار کمک می‌کند خدمت رسانی کند را با تیم تحلیل به اشتراک بگذارد. اما نکته مهمی که وجود دارد تیم توسعه نیازی ندارد به کل فضای کسب و کار مسلط باشد و تیم تحلیل نیز نیاز ندارد به کل ابزارهای محیط تکنیکال اشراف داشته باشد.

برای این اشتراک گذاری دانش دقیقاً باید چیزهایی به اشتراک گذاشته شود که برای پیاده سازی نرم افزار ضروری است. بنابراین باید یک زبان مشترک بین تیم تحلیل و تیم توسعه شکل بگیرد که شامل اصطلاحات و لغاتی باشد که بخشی از آنها مربوط به کسب و کار و قوانین کسب و کاری است و بخشی از آنها تکنیکال است.

از آنجایی که هر Bounded Context اصطلاحات کسب و کاری خود را دارد و از آنجایی که فضای تکنیکال هر Bounded Context تفاوت هایی با بقیه دارد بنابراین زبان مشترک به ازای هر Bounded Context ایجاد می‌شود.

14. تفاوت بین Data Model و Domain Model

چیست؟

برای آنکه بتوانید یک کسب و کار را در قالب یک نرم افزار پیاده سازی کنید باید تمام اجزای مهم آن را در دنیای نرم افزار مدل سازی کنید. به مدل به دست آمده Domain Model می‌گویند. این Domain Model یک سری اطلاعات به همراه خودش دارد که نیاز به نگهداری و ذخیره سازی در دیتابیس دارد. از طرفی این اطلاعاتی که باید با ساختار مشخصی در دیتابیس نگهداری شوند. به ساختاری که شکل ذخیره سازی اطلاعات در دیتابیس را مشخص می‌کند Data Model می‌گویند.

Domain Model شامل قوانین کسب و کار و رفتارهای موجودیت های کسب و کار است در حالی که Data Model فقط ساختار ذخیره سازی دیتا را مشخص می‌کند. از آنجایی که ابتدا Domain Model مشخص می‌شود و سپس Data Model با توجه به Domain Model ساخته می‌شود، این دو شبیه به یکدیگر هستند.

تمرکز Domain Model بر روی حل فضای مساله دامنه و تمرکز Data Model بر روی نحوه ذخیره سازی و فراخوانی اطلاعات در دیتابیس است.

15. تفاوت Application Logic و Domain Logic

چیست؟

Domain Logic منطق فضای مساله و کسب و کار و Application Logic منطق فضای نرم افزار است. Domain Logic شامل قوانین کسب و کار و تمام مراحل که یک فرآیند باید در کسب و کار طی شود تا یک هدف خاصی برآورده شود می‌باشد و Application Logic این مراحل را به کمک ابزار های تکنیکال و به ترتیب اجرا می‌کند.

Domain Logic برای عملکرد صحیح در دنیای نرم افزار نیاز به تکنولوژی های نرم افزار دارد اما از طرفی نباید به هیچ تکنولوژی وابسته باشد، بنابراین Application Logic به عنوان یک واسطه این کار را انجام می‌دهد.

Use Case ها از 0 تا 100 توسط Application Logic پیاده سازی می‌شوند. Application Logic به زیرساخت ها و تکنولوژی های پروژه دسترسی دارد و از طرفی می‌داند Domain Logic به چه ترتیبی باید اجرا شود. بنابراین تمام منطق کسب و کار را در لایه Domain تعریف می‌کنیم که به Domain Logic می‌گوییم و سپس تمام Use Case ها در لایه Application پیاده سازی می‌کنیم و به آن Application Logic می‌گوییم.